Task Description #1 (Password Strength Validator – Apply AI in
Security Context)

code:

```
#Apply AI to generate at least 3 assert test cases for is_strong_password(password) and
implement the validator function.
import re
def is_strong_password(password):
    if len(password) < 8:
        return False
    if not re.search(r'[A-Z]', password):
        return False
    if not re.search(r'[a-z]', password):
        return False
    if not re.search(r'[0-9]', password):
        return False
    if not re.search(r'[@$!%*?&]', password):
        return False
    if ' ' in password:
        return False
    return True
#AI-generated test cases
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
```

assert is_strong_password("ABCD@1234") == False

assert is_strong_password("Abcdefg") == False

assert is_strong_password("Abcdefg1") == False

assert is_strong_password("Abcdefg@") == False

assert is_strong_password("Abc defg1@") == False

print("All test cases passed successfully!")

#output:

All test cases passed successfully

```
PS C:\Users\ashri\OneDrive\Desktop\hh> & C:/Users/ashri/OneDrive/Desktop/hh/.venv/Scripts/python.exe "c:/Users/ashri/OneDrive/Desktop/hh/lab
 8.1.py"
All test cases passed successfully!
PS C:\Users\ashri\OneDrive\Desktop\hh>
```

Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.

#code:

#Use AI to generate at least 3 assert test cases for a

#classify_number(n) function. Implement using loops.

```
def classify_number(n):
    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"


#AI-generated test cases
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
print("All test cases passed!")
```

#OutPut:

```
PS C:\Users\ashri\OneDrive\Desktop\hh> & C:/Users/ashri/OneDrive/Desktop/hh/.venv/Scripts/python.exe "c:/Users/ashri/OneDrive/Desl
 8.1.py"
All test cases passed!
PS C:\Users\ashri\OneDrive\Desktop\hh>
```

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

• Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.

#code:

```
def is_anagram(str1, str2):
    # Remove spaces and convert to lowercase
    str1 = str1.replace(" ", "").lower()
    str2 = str2.replace(" ", "").lower()

    # Sort the characters of both strings and compare
    return sorted(str1) == sorted(str2)
# AI-generated assert test cases
assert is_anagram("listen", "silent") == True
assert is_anagram("triangle", "integral") == True
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
assert is_anagram("The eyes", "They see") == True
assert is_anagram("Astronomer", "Moon starer") == True
assert is_anagram("Conversation", "Voices rant on") == True
assert is_anagram("Hello", "Olelh") == True
assert is_anagram("Python", "Java") == False
print("All test cases passed!")
```

#output:

```
PS C:\Users\ashri\OneDrive\Desktop\hh> & C:/Users/ashri/OneDrive/Desktop/hh/.venv/Scripts/python.exe "c:/Users/ashri/OneDrive/Desl
 8.1.py"
All test cases passed!
PS C:\Users\ashri\OneDrive\Desktop\hh>
```

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

• Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

#code:

```
#Ask AI to generate at least 3 assert-based tests for an
#Inventory class with stock management

class Inventory:
    def __init__(self):
        self.stock = {}

    def add_item(self, item, quantity):
        if item in self.stock:
            self.stock[item] += quantity
        else:
            self.stock[item] = quantity

    def remove_item(self, item, quantity):
        if item in self.stock and self.stock[item] >= quantity:
            self.stock[item] -= quantity
            return True
        return False

    def get_stock(self, item):
        return self.stock.get(item, 0)
# AI-generated assert test cases
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
assert inv.remove_item("Book", 4) == False
assert inv.get_stock("Book") == 3
print("All test cases passed!")
```

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

#code:

```
#Task: Use AI to generate at least 3 assert test cases for
#validate_and_format_date(date_str) to check and convert
#dates.
#Requirements:
#o Acceptable date formats: "DD/MM/YYYY", "MM-DD-YYYY", "
YYYY.MM.DD".
#o Validate the date format and convert it to "YYYY-MM-DD".
#Example Assert Test Cases:
#assert validate_and_format_date("25/12/2024") == "2024-
12-25"
#assert validate_and_format_date("12-25-2024") == "2024-    12-25"
#assert validate_and_format_date("2024.12.25") == "2024-12-25"
#Expected Output:
#• Date validation and formatting logic passing all AI-generated test cases.
import re
from datetime import datetime
def validate_and_format_date(date_str):
    # Define regex patterns for the acceptable date formats
    patterns = [
        r'^(?P<day>\d{2})/(?P<month>\d{2})/(?P<year>\d{4})$',  # DD/MM/YYYY
        r'^(?P<month>\d{2})-(?P<day>\d{2})-(?P<year>\d{4})$',  # MM-DD-YYYY
        r'^(?P<year>\d{4})\.(?P<month>\d{2})\.(?P<day>\d{2})$'  # YYYY.MM.DD
    ]

    for pattern in patterns:
```

```
        match = re.match(pattern, date_str)
        if match:
            try:
                # Extract day, month, and year from the matched groups
                day = int(match.group('day'))
                month = int(match.group('month'))
                year = int(match.group('year'))

                # Validate the date using datetime
                datetime(year, month, day)

                # Return the formatted date
                return f"{year:04d}-{month:02d}-{day:02d}"
            except ValueError:
                continue  # If the date is invalid, try the next pattern

    raise ValueError("Invalid date format or value")
# Assert test cases
assert validate_and_format_date("25/12/2024") == "2024-12-25"
assert validate_and_format_date("12-25-2024") == "2024-12-25"
assert validate_and_format_date("2024.12.25") == "2024-12-25"
print("All test cases passed!")
```

#outPut:

```
PS C:\Users\ashri\OneDrive\Desktop\hh> & C:/Users/ashri/OneDrive/Desktop/hh/.venv/Scripts/python.exe "c:/Users/ashri/OneDrive/Desl
 8.1.py"
All test cases passed!
PS C:\Users\ashri\OneDrive\Desktop\hh>
```